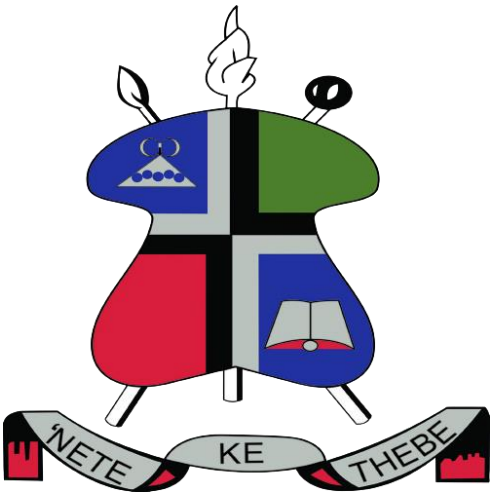


National University of Lesotho
Faculty of Science and Technology
Department of Mathematics and Computer Science

System Design - Phase 3

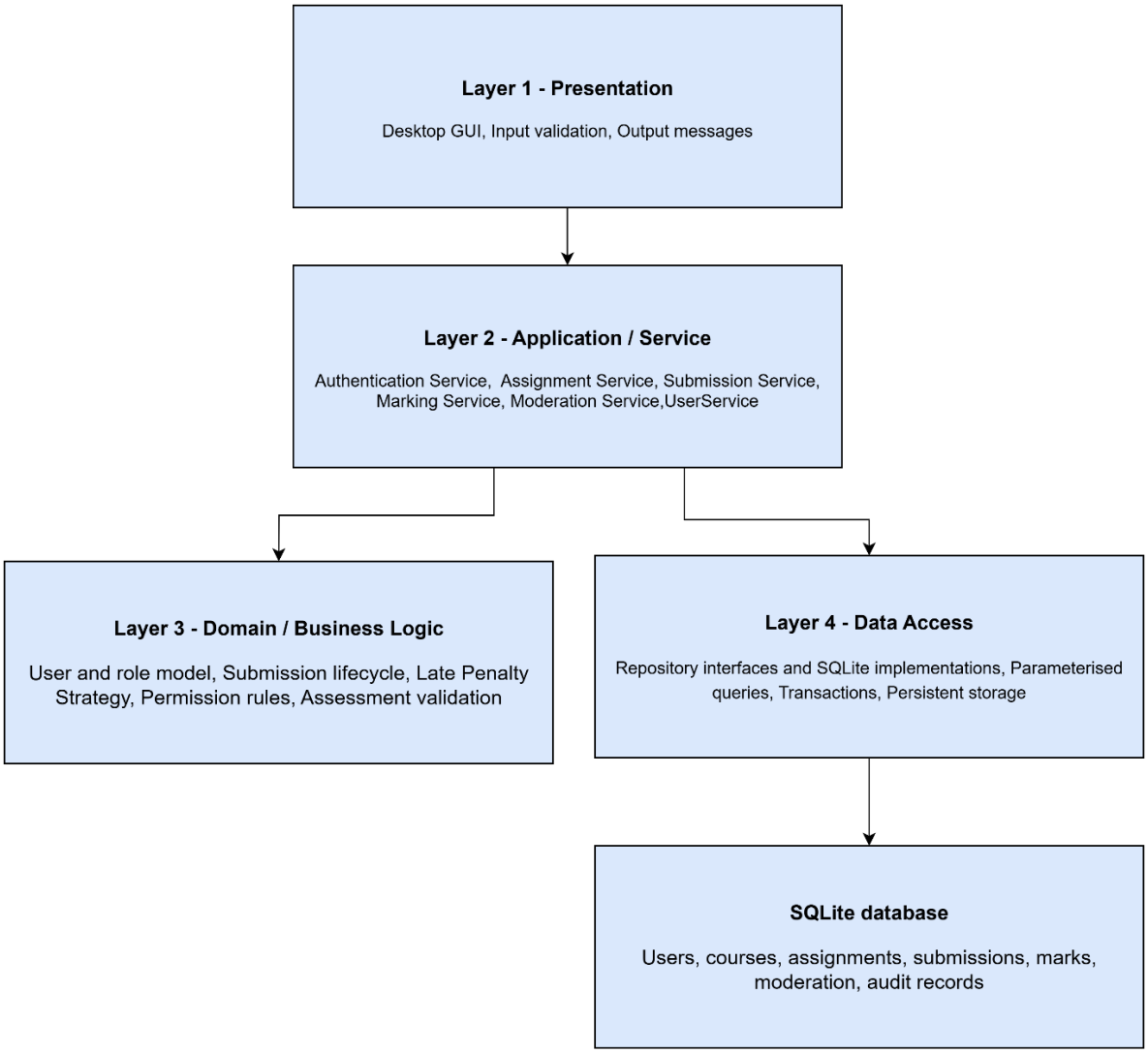
Learning Management System

Submitted by	MAFATANE. T
Student number	202321187
Course	CS4433 - Software Engineering
Submission date	18/10/2026

1. System Architecture

1.1 Layered Architecture

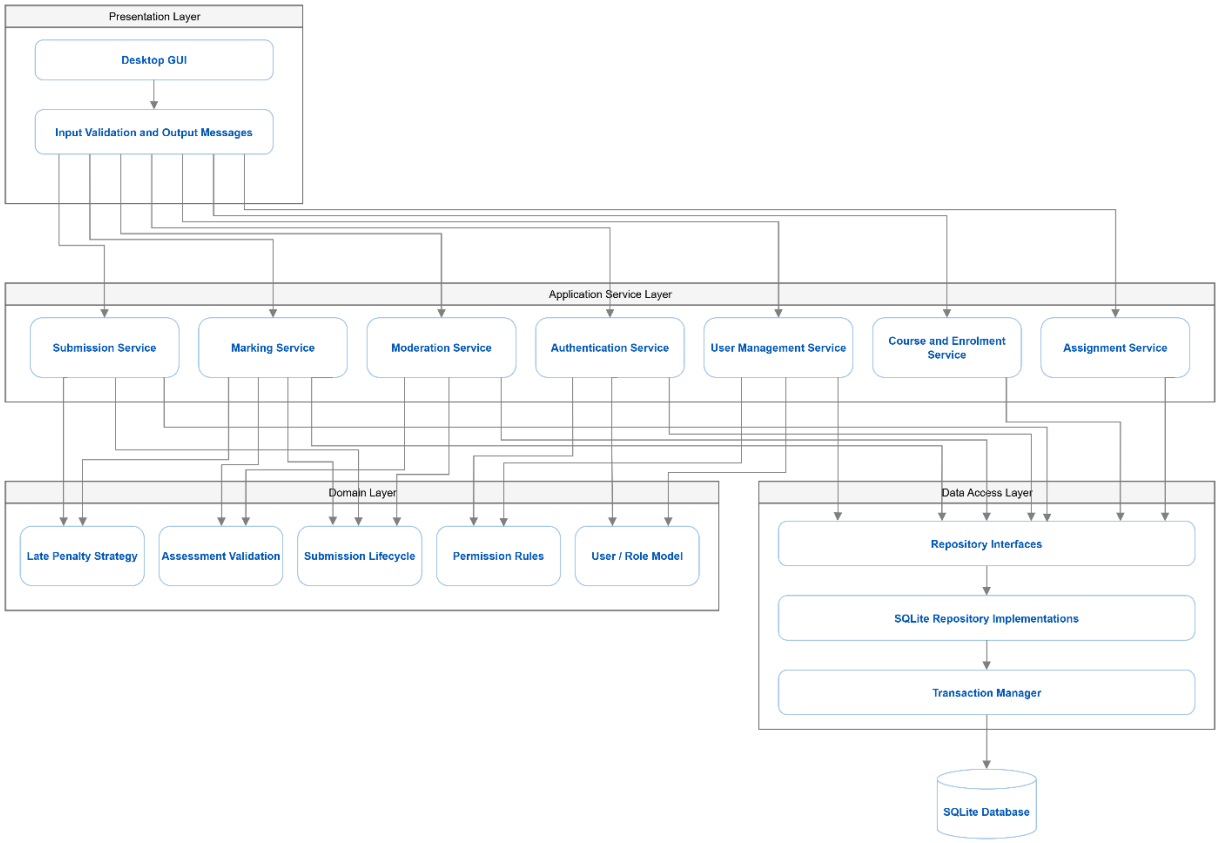
The system will use a four-layer architecture. Each layer has a distinct responsibility and communicates through defined interfaces.



Relaxed layering: the Application layer may access the Domain and Data Access layers directly. The Domain layer has no dependency on the Data Access layer.

1.2 Component diagram

The following component diagram gives a more precise view of the architecture and the main components that will be implemented.



1.3 Component responsibility

Component	**Responsibility**
Presentation UI	Collects user input, displays records, and reports errors without implementing business rules.
Authentication Service	Verifies credentials and establishes the authenticated user session.
User Management Service	Creates and manages users and their role assignments.
Course and Enrolment Service	Manages departments, courses, lecturer assignments, and student enrolments.
Assignment Service	Creates assignments, validates deadlines and maximum marks, and manages extensions.
Submission Service	Records submissions, timestamps, resubmissions, and lifecycle transitions.
Marking Service	Validates marks, applies the late-penalty strategy, stores feedback, and records mark changes.
Moderation and Release Service	Enforces HoD permissions, records moderation decisions, and releases approved marks.

Domain Layer	Encapsulates role rules, assessment policies, and submission state transitions.
Data Access Layer	Provides repository operations and executes parameterized SQLite queries within transactions.
SQLite Database	Persists users, courses, assignments, submissions, results, and audit records.

1.4 Design rationale and rejected alternative

The layered architecture was selected because it provides the following:

1. Separation of concerns: Presentation, business logic, and database operations can be developed and tested separately.
2. Maintainability: A future change to the desktop application or database implementation should not require rewriting the late-penalty rules.
3. Security: Role checks can be enforced in application services rather than relying only on hidden menu options.
4. Testability: Business services can be tested using repository interfaces and test doubles without requiring a live database for every unit test.
5. Offline operation: SQLite provides local persistent storage without requiring an external database server or internet connection.

Rejected alternative:

Three-tier client-server architecture:

A client-server architecture with a separate application server and remote database would add network configuration, server administration, and additional deployment complexity. That overhead is not justified for the specified single-computer or local-network deployment. A layered application can still be deployed on a local network if needed.

MVC design pettern:

Model-View-Controller can be useful inside the presentation layer, but it does not by itself define the database access boundary or the separation between application services and business rules. Layered architecture provides the stronger overall structure.

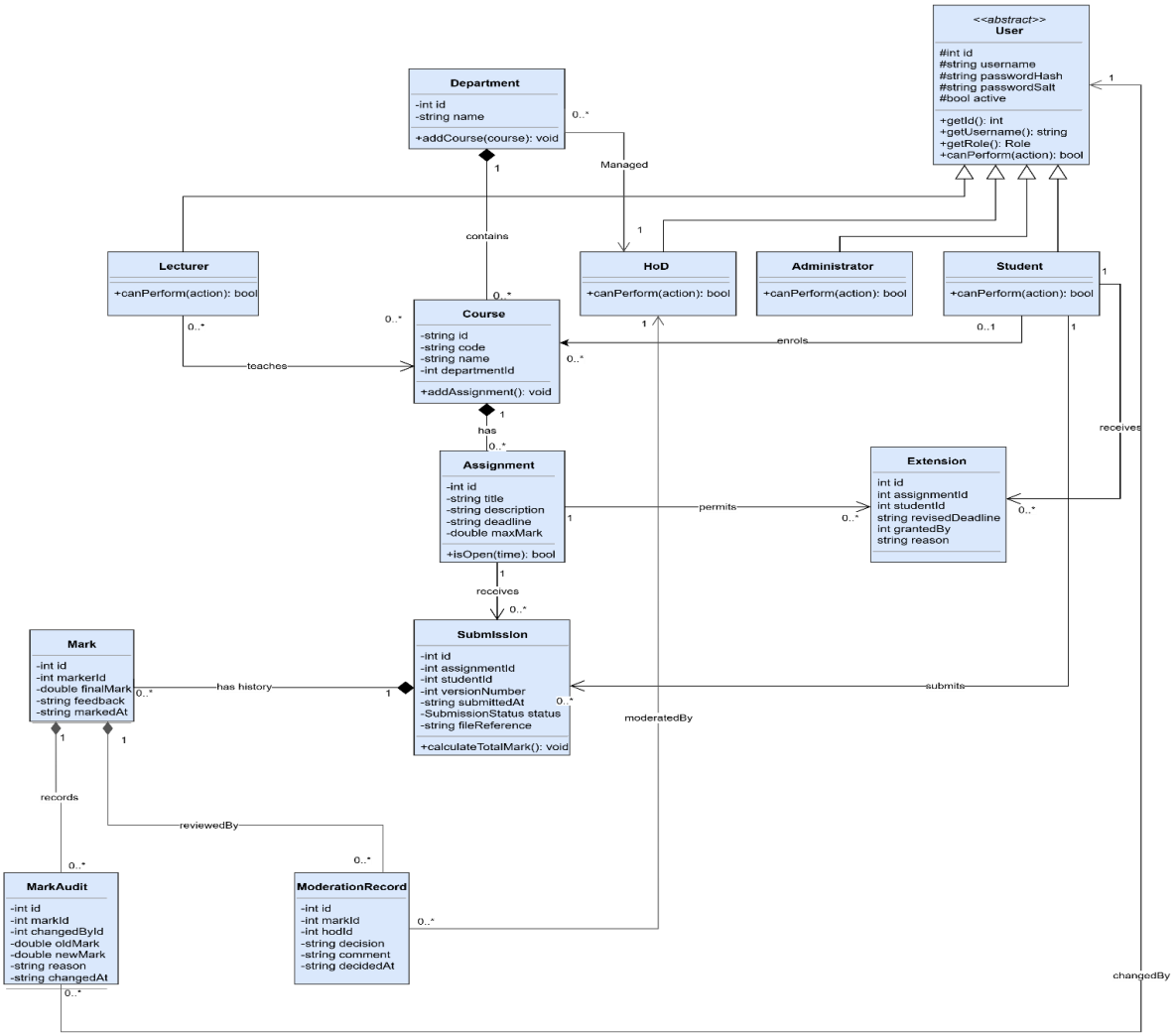
2. Class Diagram

The class diagram defines the system's classes, attributes, operations, visibility, types, inheritance, interfaces, and multiplicities. It is the blueprint for the C++ implementation.

2.1 Main domain classes and interfaces

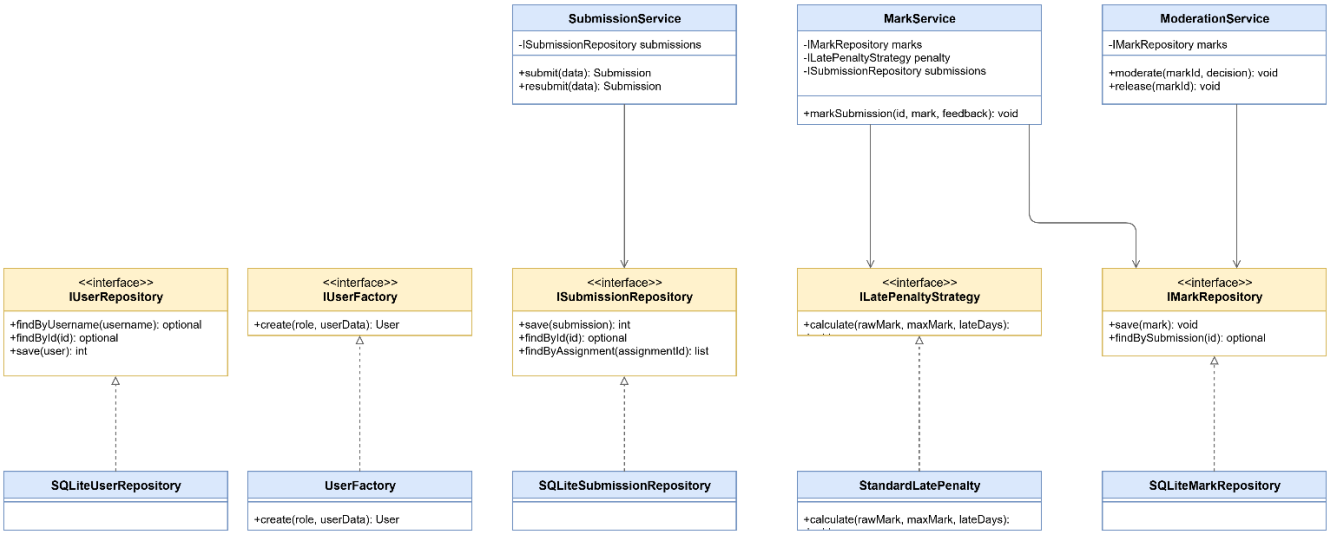
The model uses an abstract User base class with four roles. It also introduces three patterns:

- Strategy: encapsulates the late-penalty calculation.
- Repository/DAO: separates persistence from business logic.
- Factory: creates the correct role-specific user object.



2.2 Supporting interfaces and design patterns

The domain classes above are complemented by the following interfaces and service classes. They are included here separately to keep the main class diagram readable.



2.3 Design patterns and justification

Pattern	Application	Justification
Strategy	ILatePenaltyStrategy and StandardLatePenalty	Encapsulates the late-penalty algorithm so policy changes do not require rewriting marking and submission services.
Repository/DAO	IUserRepository, ISubmissionRepository, IMarkRepository and SQLite implementations	Prevents SQL and database details from spreading through the business logic; also allows mock repositories in unit tests.
Factory	IUserFactory and UserFactory	Centralizes the creation of Student, Lecturer, HoD, and Administrator objects instead of scattering role-specific construction logic throughout the application.

Include the problem, solution and pros and cons

2.4 SOLID principles

Principle	Application in the design
Single Responsibility Principle (SRP)	SubmissionService manages submission workflows, MarkingService manages marking, repositories manage persistence.
Open/Closed Principle (OCP)	A new late-penalty strategy can implement ILatePenaltyStrategy without changing the marking service.
Liskov Substitution Principle (LSP)	Every concrete user class must honor the contract of the abstract User class.
Interface Segregation Principle (ISP)	Separate repository interfaces expose only the operations required for their data responsibilities.
Dependency Inversion Principle (DIP)	Application services depend on repository and strategy interfaces rather than concrete SQLite classes or a hard-coded penalty algorithm.

2.5 Important multiplicity and integrity decisions

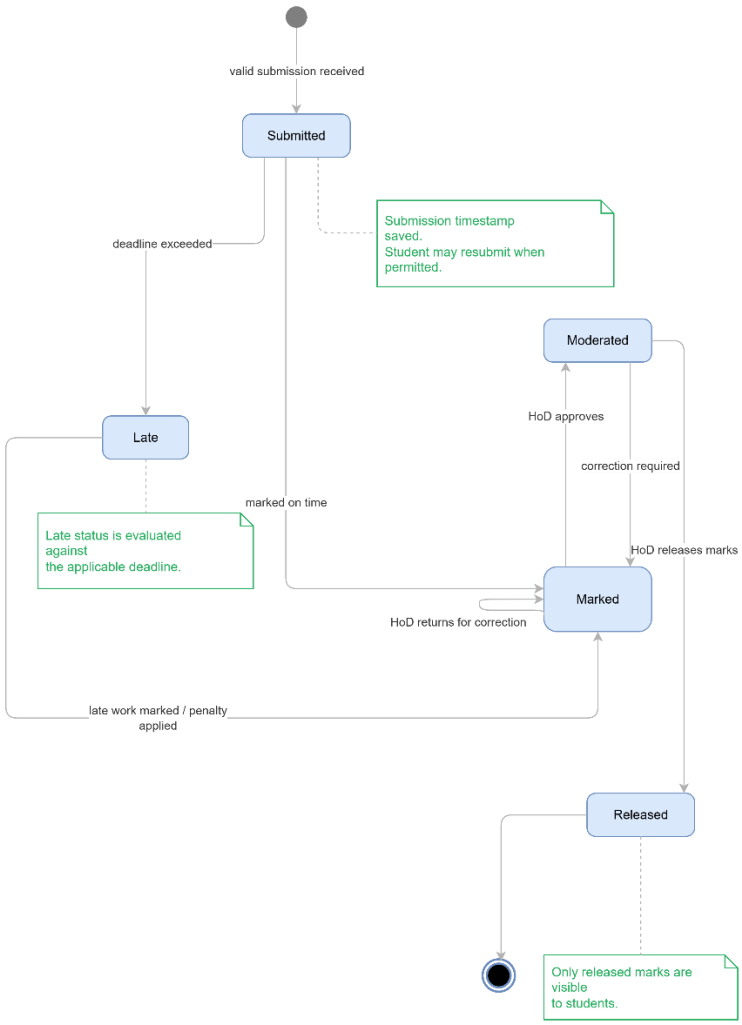
- A department can contain many courses; every course belongs to one department.
- A student can enroll in many courses, and a course can contain many students. The many-to-many relationship is implemented using Enrolment.
- Lecturers can teach multiple courses, and courses can have multiple lecturers. TeachingAssignment resolves this many-to-many relationship.
- An assignment can receive many submissions, including multiple versions from a student. The implementation must identify the latest valid submission for marking.
- A mark may have multiple audit entries and moderation records, preserving its history.

3. Behavioral models

The behavioral models describe how the system changes over time and how its components collaborate.

3.1 State machine diagram: Submission lifecycle

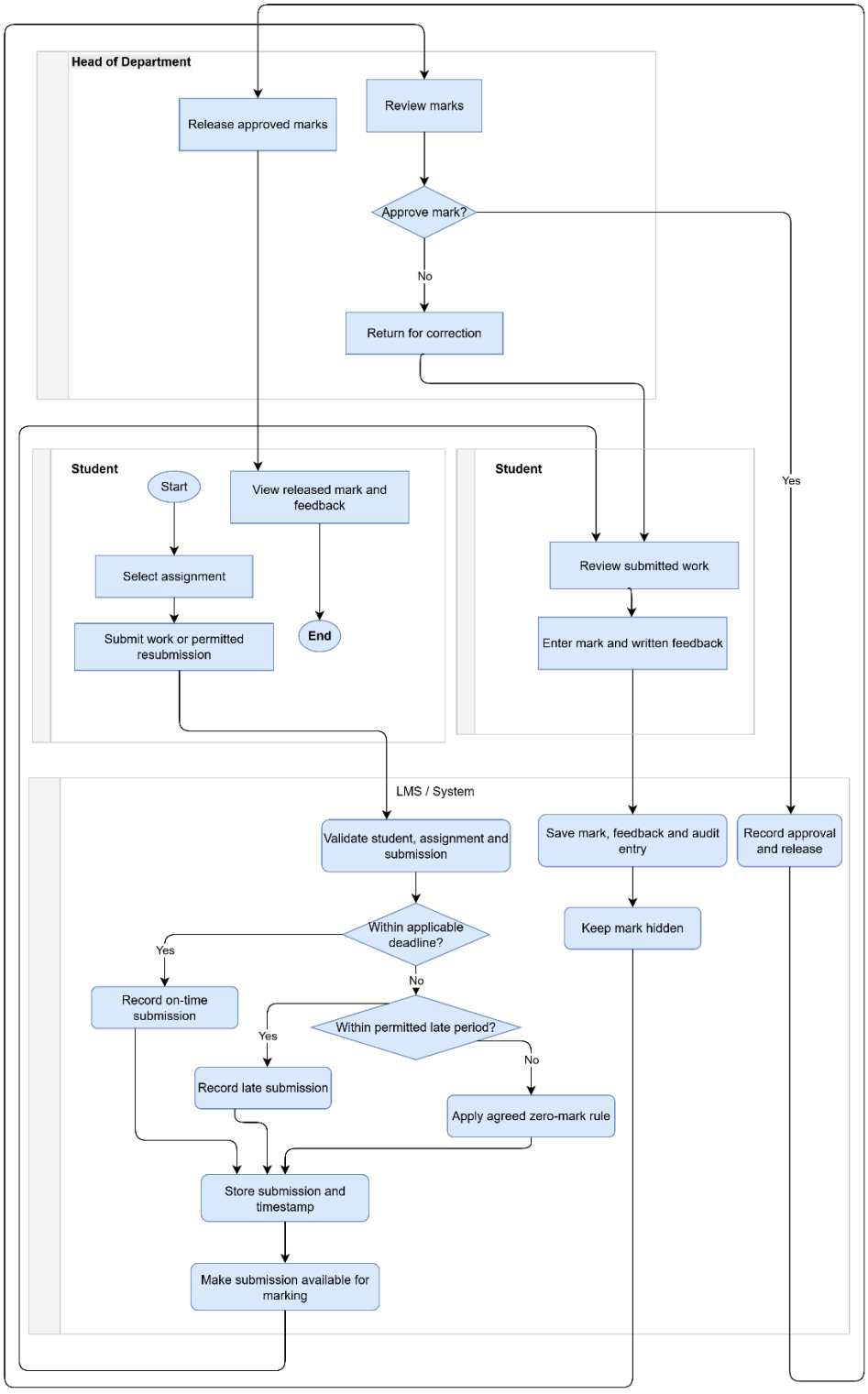
The lifecycle includes the states submitted, late, marked, moderated, and released. A returned mark must be corrected and moderated again before release.



Design note: The submission status and mark status are related but not identical. In the database design, a submission can have a lifecycle status while a separate moderation record records approval or rejection. This avoids treating a mark correction as if the original submitted work never existed.

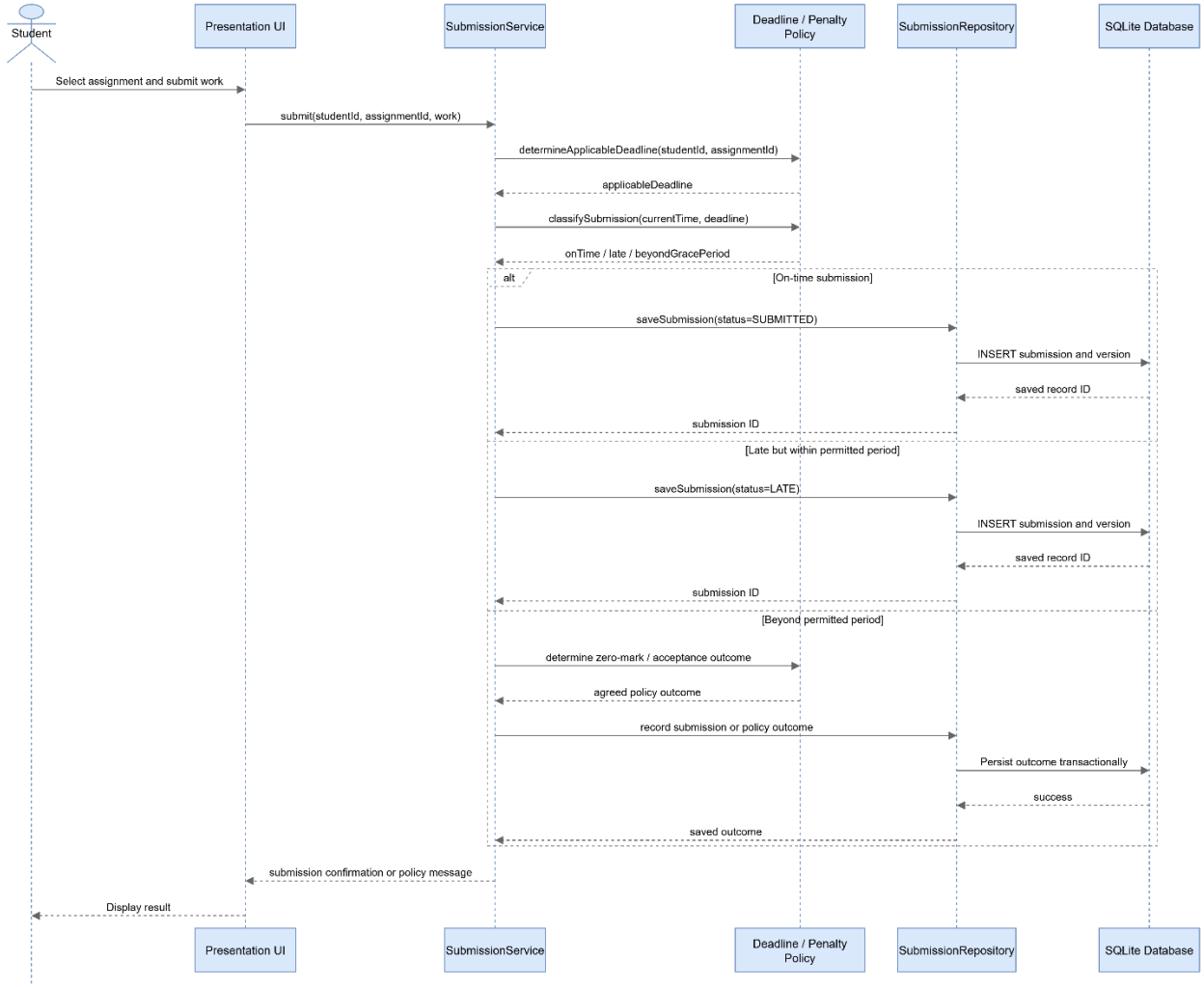
The state machine focuses on the principal assessment lifecycle. Resubmission creates a new version of a submission and must not silently erase the earlier submission record.

3.2 Activity diagram: Submission to released mark

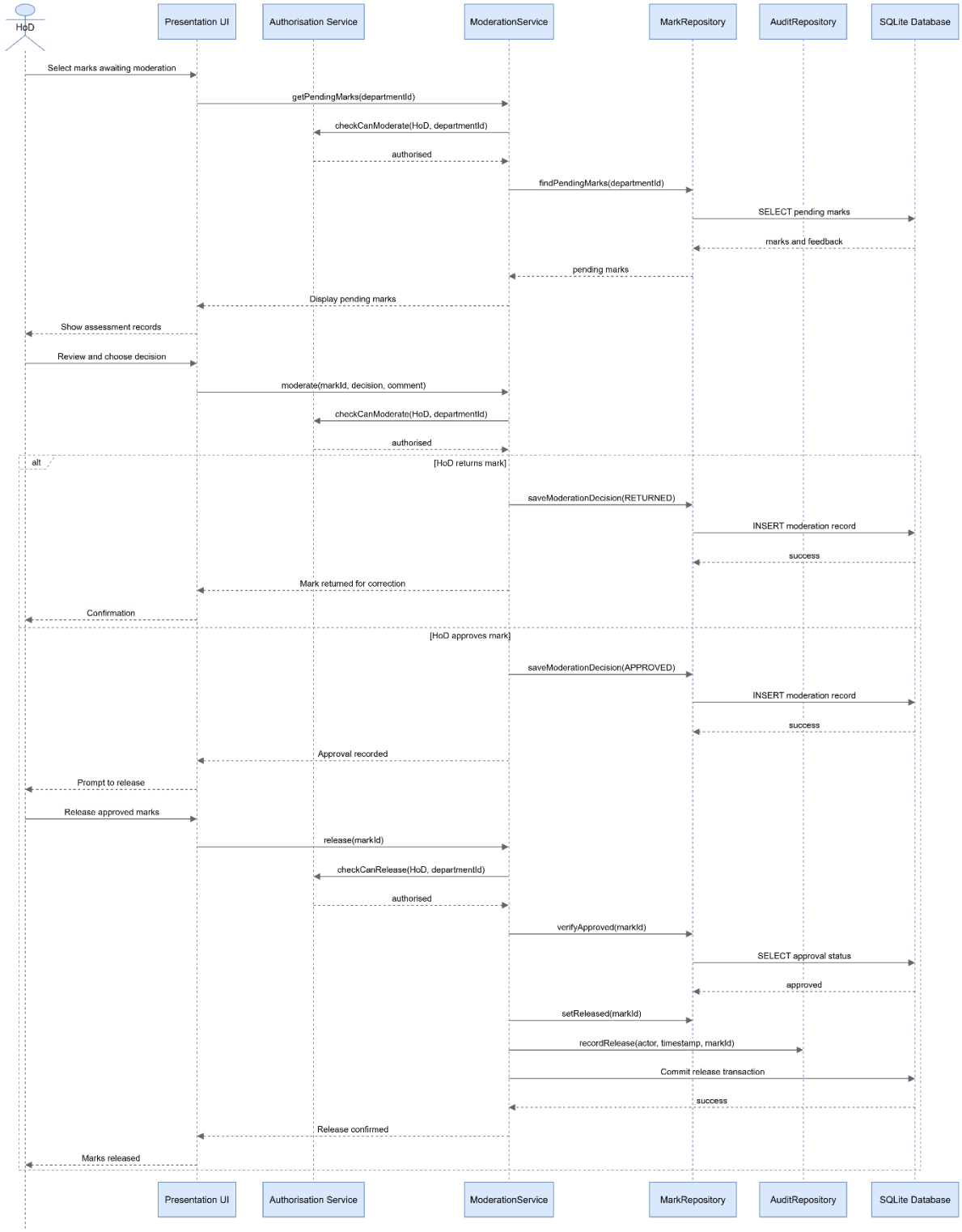


3.3 Sequence diagram: Submit Assignment

This diagram models the normal submission path, a late submission, and a resubmission.



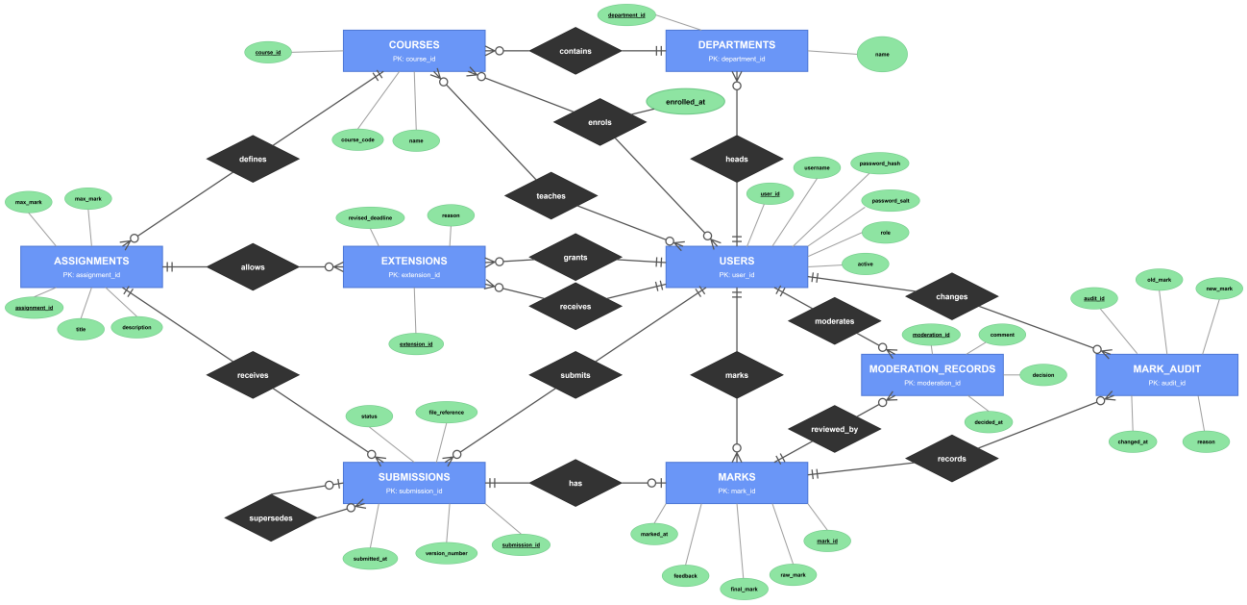
3.4 Sequence diagram: Moderate and Release Marks



4. Database design

4.1 Entity Relationship Diagram (ERD)

The following ERD gives a high-level view of the system's principal entities, their attributes, and the relationships between them.



4.2 Data dictionary

The following dictionary documents the important attributes and their constraints. PK means primary key, FK means foreign key, and UK means unique key.

Table	Attribute	Type	Constraint / meaning
USERS	user_id	INTEGER	PK; generated account identifier.
USERS	username	TEXT	UK, NOT NULL; login name.
USERS	password_hash	TEXT	NOT NULL; salted password hash, never plaintext.
USERS	password_salt	TEXT	NOT NULL; salt used by the password-hashing implementation.

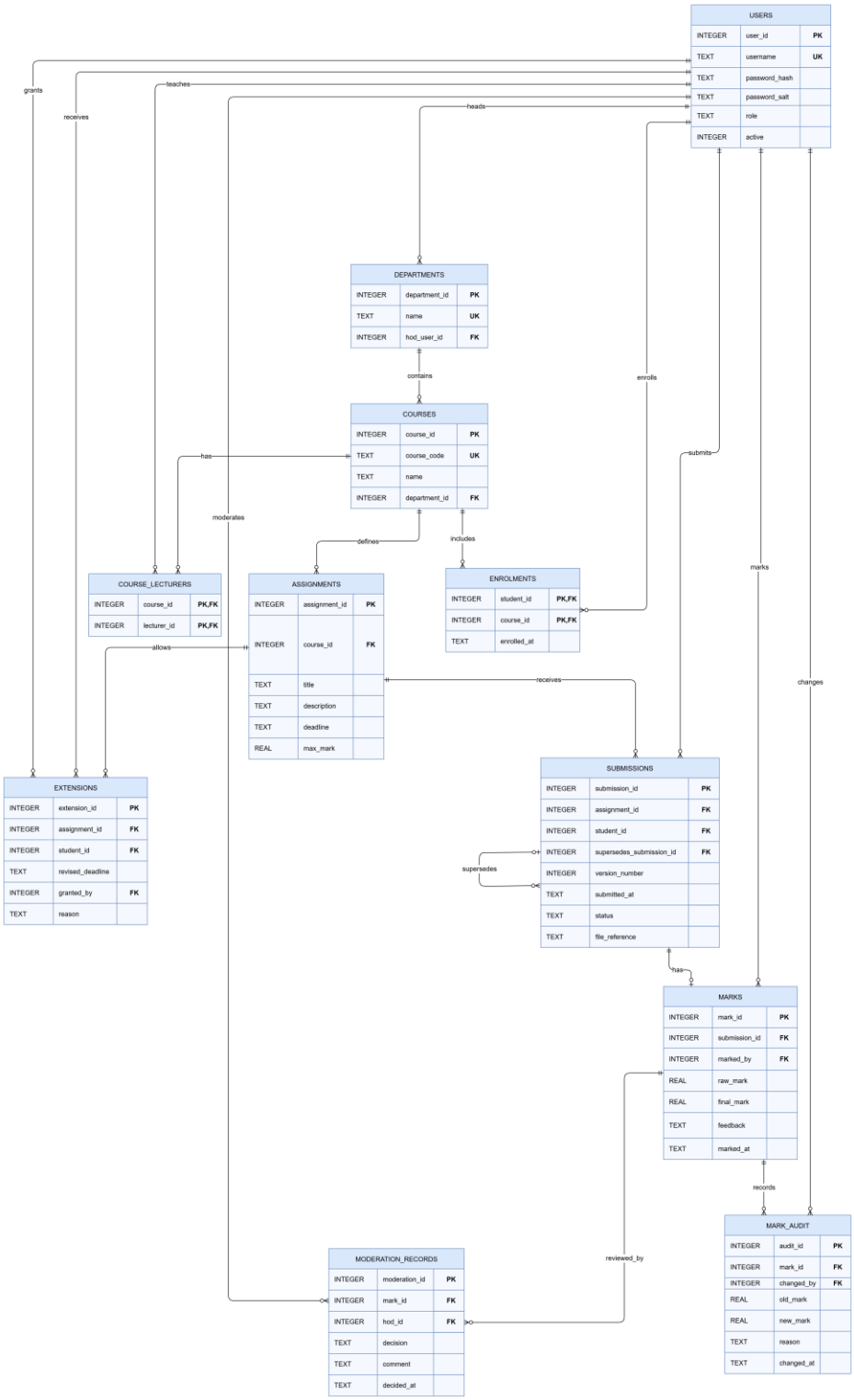
USERS	role	TEXT	NOT NULL; restricted to the four authorized roles.
USERS	active	INTEGER	NOT NULL; 1 = active, 0 = inactive.
DEPARTMENTS	department_id	INTEGER	PK.
DEPARTMENTS	name	TEXT	UK, NOT NULL; department name.
DEPARTMENTS	hod_user_id	INTEGER	FK to USERS; HoD responsible for the department.
COURSES	course_id	INTEGER	PK.
COURSES	course_code	TEXT	UK, NOT NULL; unique course identifier.
COURSES	name	TEXT	NOT NULL; course name.
COURSES	department_id	INTEGER	FK to DEPARTMENTS.
COURSE_LECTURERS	course_id	INTEGER	Composite PK; FK to COURSES.
COURSE_LECTURERS	lecturer_id	INTEGER	Composite PK; FK to USERS; assigned lecturer.
ENROLMENTS	student_id	INTEGER	Composite PK; FK to USERS.
ENROLMENTS	course_id	INTEGER	Composite PK; FK to COURSES.
ENROLMENTS	enrolled_at	TEXT	NOT NULL; enrolment timestamp.
ASSIGNMENTS	assignment_id	INTEGER	PK.
ASSIGNMENTS	course_id	INTEGER	FK to COURSES.
ASSIGNMENTS	title	TEXT	NOT NULL; assignment title.
ASSIGNMENTS	description	TEXT	Assignment instructions.
ASSIGNMENTS	deadline	TEXT	NOT NULL; ISO 8601 date/time.
ASSIGNMENTS	max_mark	REAL	NOT NULL; must be greater than zero.
EXTENSIONS	extension_id	INTEGER	PK.
EXTENSIONS	assignment_id	INTEGER	FK to ASSIGNMENTS.
EXTENSIONS	student_id	INTEGER	FK to USERS; student receiving extension.

EXTENSIONS	revised_deadline	TEXT	NOT NULL; approved revised deadline.
EXTENSIONS	granted_by	INTEGER	FK to USERS; lecturer granting extension.
EXTENSIONS	reason	TEXT	NOT NULL; reason for extension.
SUBMISSIONS	submission_id	INTEGER	PK; identifier for one submission version.
SUBMISSIONS	assignment_id	INTEGER	FK to ASSIGNMENTS.
SUBMISSIONS	student_id	INTEGER	FK to USERS.
SUBMISSIONS	supersedes_submission_id	INTEGER	Optional self-referencing FK to previous version.
SUBMISSIONS	version_number	INTEGER	NOT NULL; version sequence number.
SUBMISSIONS	submitted_at	TEXT	NOT NULL; submission timestamp.
SUBMISSIONS	status	TEXT	NOT NULL; submission lifecycle state.
SUBMISSIONS	file_reference	TEXT	NOT NULL; path or identifier for stored work.
MARKS	mark_id	INTEGER	PK.
MARKS	submission_id	INTEGER	FK, UNIQUE; at most one current mark record per submission.
MARKS	marked_by	INTEGER	FK to USERS; lecturer who marked the work.
MARKS	raw_mark	REAL	NOT NULL; mark before penalty, within the assignment's mark range.
MARKS	final_mark	REAL	NOT NULL; final mark after penalty.
MARKS	feedback	TEXT	NOT NULL; written feedback.
MARKS	marked_at	TEXT	NOT NULL; marking timestamp.
MODERATION_RECORDS	moderation_id	INTEGER	PK.

MODERATION_RECORDS	mark_id	INTEGER	FK to MARKS.
MODERATION_RECORDS	hod_id	INTEGER	FK to USERS; HoD making the decision.
MODERATION_RECORDS	decision	TEXT	NOT NULL; APPROVED or RETURNED.
MODERATION_RECORDS	comment	TEXT	Moderation comment or correction reason.
MODERATION_RECORDS	decided_at	TEXT	NOT NULL; decision timestamp.
MARK_AUDIT	audit_id	INTEGER	PK.
MARK_AUDIT	mark_id	INTEGER	FK to MARKS.
MARK_AUDIT	changed_by	INTEGER	FK to USERS.
MARK_AUDIT	old_mark	REAL	Previous mark; NULL for initial creation.
MARK_AUDIT	new_mark	REAL	NOT NULL; new mark.
MARK_AUDIT	reason	TEXT	NOT NULL; reason for the change.
MARK_AUDIT	changed_at	TEXT	NOT NULL; change timestamp.

4.3 SQL Schema

The following schema is designed for SQLite. It uses primary keys, foreign keys, uniqueness constraints, check constraints, and indexes to enforce the data-integrity



4.4 Conclusion

The proposed design uses a layered architecture to separate the user interface, application services, domain rules, and database access. Its class model applies inheritance and polymorphism for the four user roles, while the Strategy, Repository/DAO, and Factory patterns support maintainability and testability.

The behavioral models define the assessment lifecycle and the collaboration required to submit, mark, moderate, and release work. The SQLite schema provides persistent storage for the main domain entities, including submission versions, individual extensions, moderation decisions, and mark-change history.